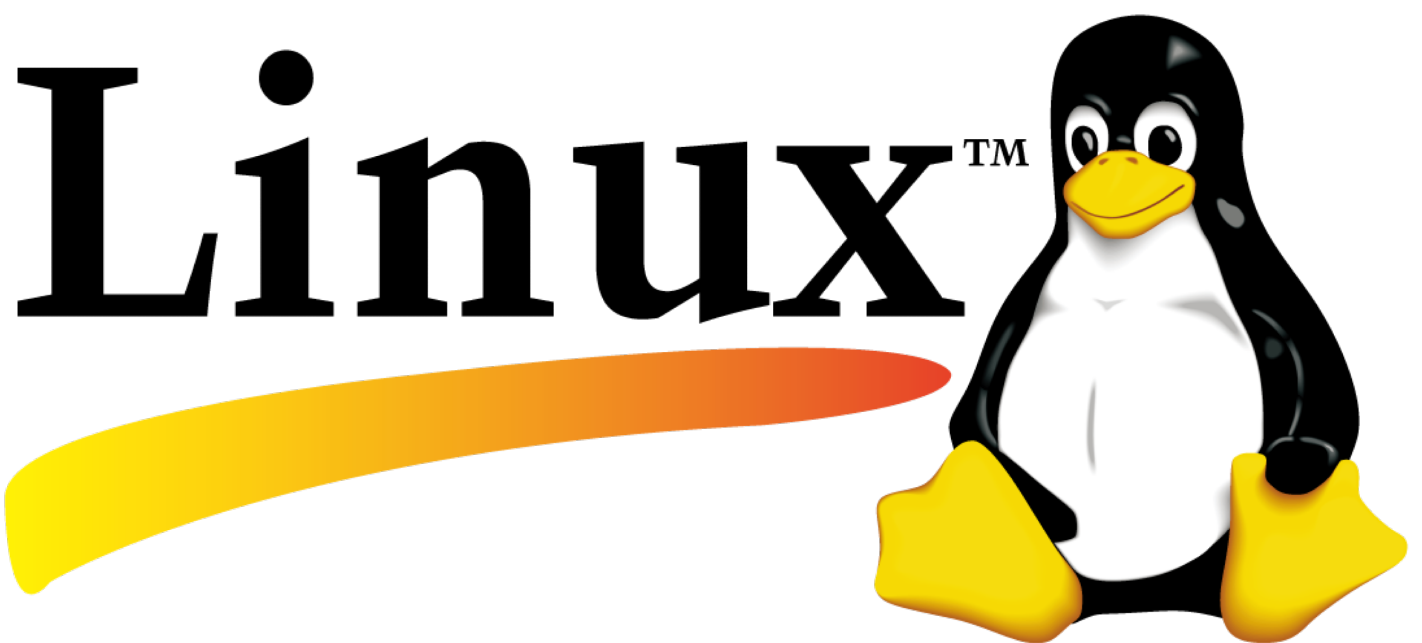




Appunti

Stacchiotti Elia

# Indice

|          |                            |           |
|----------|----------------------------|-----------|
| <b>1</b> | <b>Git e GitHub</b>        | <b>1</b>  |
| 1.1      | Git . . . . .              | 1         |
| 1.2      | I Branch . . . . .         | 5         |
| 1.3      | Merge dei branch . . . . . | 7         |
|          | <b>Fonti</b>               | <b>10</b> |

Listati

# 1

## Git e GitHub

In questo capitolo vengono introdotti due strumenti fondamentali nello sviluppo software moderno: **Git** e **GitHub**. Essi sono ampiamente utilizzati, soprattutto nel contesto open source, dove per definizione l'intera community è invitata a contribuire attivamente ai progetti.

Sebbene spesso vengano utilizzati in modo coordinato, è importante distinguerne chiaramente i ruoli. **Git è un sistema di controllo di versione distribuito, che consente di tracciare nel tempo le modifiche ai file, mantenendo una cronologia completa del progetto.** In altre parole, Git permette di effettuare il *versioning*, rendendo possibile tornare a versioni precedenti, confrontare modifiche e lavorare in parallelo su diverse funzionalità. **GitHub**, invece, è una **piattaforma online che consente di ospitare repository Git su un server remoto, rendendoli accessibili tramite Internet**. Questo facilita la collaborazione tra più sviluppatori, offrendo strumenti aggiuntivi come la gestione delle modifiche, il tracciamento dei problemi e la revisione del codice. Mentre le funzionalità di hosting offerte da GitHub possono essere sostituite da altri servizi simili (o anche da server privati), il ruolo di **Git come sistema di versionamento rimane centrale ed è difficilmente rimpiazzabile**.

Nei paragrafi successivi verranno analizzati separatamente Git e GitHub, partendo dai concetti fondamentali fino ad arrivare al loro utilizzo combinato.

### 1.1. Git

Il *versioning* dei file è una pratica che permette di mantenere una cronologia delle modifiche rendendo possibile in qualsiasi momento risalire a uno stato precedente dei file. Un modo rudimentale per ottenere questo risultato consiste nel creare copie complete dell'intero archivio ogni volta che vengono effettuate delle modifiche e conservarle per eventuali ripristini futuri. Sebbene questo approccio possa sembrare inizialmente funzionale, presenta diverse limitazioni, in primo luogo la dimensione dell'archivio cresce rapidamente: più i file sono grandi, più le copie richiedono spazio di archiviazione, in secondo luogo, in un contesto condiviso tra più utenti, avere molte copie indipendenti (non collegate tra loro) rende difficile tracciare le modifiche effettuate e comprendere come il progetto si sia evoluto nel tempo. Infine, anche la sincronizzazione su sistemi cloud risulta inefficiente, specialmente in presenza di archivi di grandi dimensioni.

Git nasce proprio per risolvere questi problemi, introducendo un sistema di versionamento efficiente e strutturato. A differenza dei sistemi di controllo di versione centralizzati che si basano su un server remoto per gestire e sincronizzare le modifiche tra più utenti, **Git è un sistema distribuito ovvero l'intera cronologia del progetto è salvata in locale su ogni macchina**, questo approccio offre diversi vantaggi: è possibile lavorare completamente offline sull'archivio, avendo la possibilità di consultare la cronologia, ripristinare versioni precedenti dei file e continuare lo sviluppo senza dipendere dalla connessione a Internet. Un altro aspetto fondamentale riguarda il modo in cui Git memorizza le modifiche infatti **le dimensioni di un archivio Git rimangono generalmente contenute grazie a un sistema basato su istantanee (snapshots)**. Per comprendere questo concetto, è utile fare un confronto con altri sistemi di versionamento in particolare quelli che usano un approccio basato sulle differenze

dove sostanzialmente viene salvata una versione iniziale dei file e, successivamente, solo le modifiche (dette *diff*) rispetto alla versione precedente. L'approccio **per istantanee usato da Git prevede invece che ogni volta che viene salvata una nuova versione (commit), il sistema registra un'istantanea completa dello stato dei file in quel momento** (per evitare sprechi di spazio, Git non duplica realmente tutti i file, se un file non è stato modificato, viene semplicemente mantenuto un riferimento alla versione già esistente). Riassunto in 2 righe, **Git riesce a combinare i vantaggi di una rappresentazione completa dello stato del progetto con un utilizzo efficiente dello spazio.**

Iniziamo ora ad analizzare in modo più pratico come funziona Git e come utilizzarlo. Una cartella gestita da Git è facilmente riconoscibile dalla presenza della directory `.git`, che contiene tutte le informazioni necessarie al versionamento del progetto, inclusa la cronologia delle versioni. Accanto a questa cartella possono essere presenti altri file, essi rappresentano lo stato corrente del progetto, ovvero l'ultima versione salvata (detta **commit**) su cui è possibile lavorare. Questi file possono essere modificati, eliminati o aggiunti e una volta completate le modifiche, è possibile registrare una nuova versione del progetto utilizzando un comando Git (`git commit`, successivamente spiegato). La costruzione di una versione prevede che ogni file possa trovarsi in uno dei seguenti tre stati:

- **Committed:** il file è presente nell'ultima versione del progetto, più in generale lo stato del file è riconducibile ad una delle versioni del progetto.
- **Modified:** il file è stato modificato rispetto alla versione di partenza ma le modifiche non sono ancora state confermate per essere inserite nella nuova versione che verrà creata.
- **Staged:** il file è stato selezionato per essere incluso nella prossima versione. In questo stato, Git "fotografa" lo stato corrente del file che costituirà la nuova versione.

Questi tre stati corrispondono a tre aree logiche in cui è possibile suddividere il funzionamento di Git:

- **Working Directory:** contiene i file su cui si lavora attivamente, è un concetto indipendente dagli stati che assumono i file, quindi nella *Working Directory* sono presenti file *committed*, *modified* e *staged*.
- **Staging Area:** contiene, come istantanee, i file pronti per essere inclusi nella prossima versione.
- **Repository:** contiene, come istantanee, i file presenti nelle versioni salvate (commit) del progetto.

In termini operativi il flusso di lavoro tipico è il seguente: si parte con l'avere tutti file *committed* appartenenti all'area *Repository*, eventualmente si modificano (essendo che fanno parte della *Working Directory*, passano allo stato *modified*) poi si selezionano le modifiche da includere nella nuova versione collocando lo stato dei file (stato inteso come contenuto) nella *Staging Area*, in quel frangente il file è nello stato *staged*, infine, con un commit, si salvano definitivamente tutte le istantanee dei file nella *Staging Area* nel *Repository* che di fatto diventano *committed*, l'immagine nella figura 1.1 descrive il workflow appena descritto.

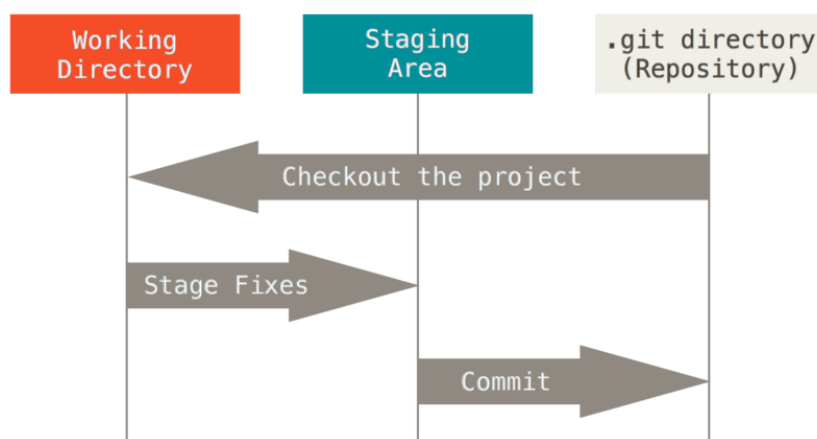


Figura 1.1: Tipico workflow git per effettuare il versioning di file

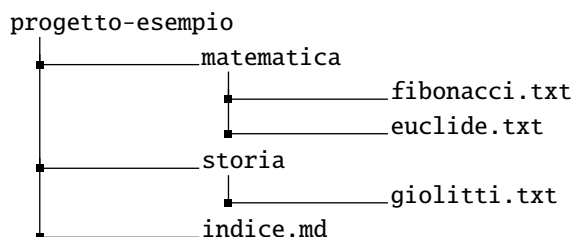
Dopo aver visto come funziona Git per quanto riguarda la creazione di una versione vediamo adesso come si fa in pratica, cioè a livello di comandi. Si può iniziare creando una cartella nel sistema, apriamo poi Git Bash all'interno di essa e digitiamo il comando `git init`, questo crea nella la cartella `.git`, si può notare anche che compare la scritta `master` accanto al percorso, successivamente sarà maggiormente chiaro ma tale scritta identifica con precisione una particolare versione che è poi quella su cui stiamo lavorando in termini di file. Digitiamo poi i comandi `git config -global user.name < nome >` e `git config -global user.email < email >` questi servono per impostare l'identità di chi effettua i commit (quindi le versioni), sono semplici stringhe e non esiste un processo di autenticazione, l'opzione `-global` imposta come globali le configurazioni che andiamo a svolgere quindi dopo questi comandi, qualsiasi cartella sottoposta a versionamento avrà versioni firmate con `< nome >` e `< email >` inseriti. Con il comando `git config -list` si può vedere tutta la configurazione Git attiva per la particolare cartella, leggendo in corrispondenza della riga `core.editor= <nome editor attivo>` è possibile vedere quale editor interno Git utilizza, per cambiare "globalmente" questa impostazione si usa il comando `git config -global core.editor < nome editor >` dove `< nome editor >` deve essere tra quelli preinstallati nella versione di Git, tra i più comuni ci sono *nano* e *VIM*

La cartella sottoposta a versionamento, allo stato attuale, non contiene file effettivi del progetto (i file presenti in `.git` non rappresentano contenuto dell'archivio, ma sono utilizzati esclusivamente da Git per gestire il versionamento); è quindi possibile creare alcuni file di esempio. In questo caso si consideri la struttura riportata in figura 1.2, dove i file `.txt` contengono semplice testo, mentre il file `.md` presenta una struttura del tipo:

```
# matematica
- fibonacci
- euclide

# storia
- giolitti
```

Figura 1.2: Struttura della cartella progetto-esempio versionata con Git



A questo punto si eseguono i comandi `git add *` e successivamente `git commit -m "initial commit"`; la sequenza di questi due comandi crea la prima versione della cartella `progetto-esempio`, ovvero il primo commit, che per convenzione viene spesso chiamato *initial commit*. I file presenti nella cartella, visualizzabili ad esempio tramite il comando `ll`, risultano ora salvati nel repository e si trovano quindi nello stato *committed*. Si procede effettuando alcune modifiche, ad esempio si può modificare il file `euclide.txt` aggiungendo la parola "Teorema" e il file `fibonacci.txt` eliminando una parola qualsiasi, in seguito a queste operazioni, i file passano dallo stato *committed* allo stato *modified*. Eseguendo il comando `git diff` è possibile visualizzare le differenze tra la versione salvata nel repository e quella attuale nella working directory (in realtà di default il comando `diff` confronta i file che si trovano nella staging area e working directory, tuttavia se la staging area non viene modificata, e questo accade dopo ogni commit prima di eseguire il comando `git add`, allora questa contiene solo i file *committed*), a seguire è riportato l'output per questo esempio:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git diff
warning: in the working copy of 'matematica/euclide.txt', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'matematica/fibonacci.txt', LF will be replaced by CRLF
the next time Git touches it
diff --git a/matematica/euclide.txt b/matematica/euclide.txt
```

```

index fd8eea5..c46511f 100644
--- a/matematica/euclide.txt
+++ b/matematica/euclide.txt
@@ -1 +1,2 @@
Euclide padre della geometria.
+Teorema
diff --git a/matematica/fibonacci.txt b/matematica/fibonacci.txt
index 5f8a94b..ebd3924 100644
--- a/matematica/fibonacci.txt
+++ b/matematica/fibonacci.txt
@@ -1 +1 @@
-Fibonacci matematico Pisano. La successione aurea.
+Fibonacci matematico. La successione aurea.

```

Sono presenti varie informazioni tecniche come le righe `diff -git ...`, `index ...`, ecc.), che servono a identificare i file e le versioni coinvolte nel confronto, ma che possono essere inizialmente trascurate, gli elementi fondamentali sono invece le righe precedute dai simboli:

- -, che indica una riga presente nella versione precedente ma rimossa
- +, che indica una riga aggiunta nella versione modificata

Nel caso dell'esempio, si osserva che nel file `euclide.txt` è stata aggiunta la riga `Teorema`, mentre nel file `fibonacci.txt` è stata modificata una riga tramite la rimozione di una parola (infatti nella versione attuale manca "Pisano"). Oltre a rilevare le modifiche, Git segnala che esse non sono ancora state incluse nella prossima versione; infatti, eseguendo il comando `git status -u`, si ottiene un output che indica chiaramente come i file modificati non siano ancora stati aggiunti alla staging area, output del comando a seguire:

```

snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
  modified:   matematica/euclide.txt
  modified:   matematica/fibonacci.txt

no changes added to commit (use "git add" and/or "git commit -a")

```

Git evidenzia che i file `euclide.txt` e `fibonacci.txt` sono nello stato *modified*, ma non ancora nello stato *staged*, e quindi non verranno inclusi nel prossimo commit. Se si desidera includere nel prossimo commit lo stato attuale del file `euclide.txt`, è possibile utilizzare il comando `git add matematica/euclide.txt`; in questo modo il file passa allo stato *staged*. Eseguendo nuovamente `git status`, si osserva che tale file è ora pronto per essere incluso nella nuova versione, mentre gli altri file rimangono invariati rispetto all'ultimo commit. Si può quindi eseguire il comando `git commit -m "modificato il file euclide"`, creando una nuova versione del progetto. A questo punto, la working directory contiene sia file aggiornati all'ultima versione (committed), sia file che presentano modifiche non ancora salvate, come nel caso di `fibonacci.txt`. Utilizzando nuovamente `git diff`, si nota che le differenze relative a `fibonacci.txt` sono le stesse di prima, poiché il confronto avviene sempre rispetto all'ultima versione salvata nel repository (che è la stessa della prima versione). Può sembrare quindi che un file sia contemporaneamente committed e modified, ma in realtà si stanno considerando due versioni diverse dello stesso file: quella salvata nel repository e quella attualmente modificata nella working directory. Possiamo proseguire con l'esercizio aggiungendo il file `fibonacci.txt` alla Staging area quindi facciamo `git add matematica/fibonacci.txt`, a questo punto modifichiamo ancora una volta il file `fibonacci.txt`, aggiungiamo la parola "Spirale", adesso eseguiamo il comando `git status`, ecco l'output:

```

snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
  modified:   matematica/fibonacci.txt

```

```
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified:   matematica/fibonacci.txt
```

vediamo come lo stesso file risulti sia *modified* che *staged* ma ancora una volta non è così perchè quelli che si stanno considerando sono per Git due file diversi essendo che hanno un contenuto diverso. Facendo `git diff` ora si vedono chiaramente le differenze tra la versione del file *staged* e la versione del file *modified* cioè quello nella working directory, con il comando `git diff --staged` invece si vedono le differenze tra la versione del file *committed* (quello nell'area repository) e la versione del file *modified*, ancora, quello nella working directory, per completezza lascio sotto i due output.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git diff
warning: in the working copy of 'matematica/fibonacci.txt', LF will be replaced by CRLF
the next time Git touches it
diff --git a/matematica/fibonacci.txt b/matematica/fibonacci.txt
index ebd3924..3bf46c9 100644
--- a/matematica/fibonacci.txt
+++ b/matematica/fibonacci.txt
@@ -1,1 @@
-Fibonacci matematico. La successione aurea.
+Fibonacci matematico. La successione aurea. Spirale
```

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git diff --staged
diff --git a/matematica/fibonacci.txt b/matematica/fibonacci.txt
index 5f8a94b..ebd3924 100644
--- a/matematica/fibonacci.txt
+++ b/matematica/fibonacci.txt
@@ -1,1 @@
-Fibonacci matematico Pisano. La successione aurea.
+Fibonacci matematico. La successione aurea.
```

Possiamo procedere effettuando un commit, si può facilmente verificare che la versione divenuta *committed* del file `fibonacci.txt` è quella che prima era stata inserita nella staging area. Quest'ultimo esempio è utile per far capire che Git tratta i file come istantanee ovvero quando si aggiungono file alla staging area è come se questi venissero "fotografati" e tale fotografia sarà presente nel successivo commit che si può allora intendere come una raccolta di "foto".

Con queste nozioni è possibile solo proseguire in avanti ovvero si possono continuare a creare versioni della cartella usando Git ma senza beneficiare di nessun "vantaggio", tuttavia questa prima parte era maggiormente pensata per dare un'idea del concetto di versione in Git.

## 1.2. I Branch

Per comprendere il funzionamento dei branch in Git è utile introdurre una rappresentazione grafica della cronologia dei commit; a tal fine, il comando `git log --all --oneline --graph` consente di visualizzare la sequenza delle versioni sotto forma di una struttura ad albero, mettendo in evidenza come i commit siano collegati tra loro. A seguire è riportato un output di esempio in cui praticamente tutte le versioni del progetto sono, per ora, allineate su un unico ramo.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git log --all --oneline --graph
* 8f32569 (HEAD -> master) modificato il file giolitti
* 2877e77 modificato il file giolitti
* 987e4ba aggiornato file fibonacci
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

Si può osservare che ogni commit è identificato in modo univoco da un hash, utilizzando l'opzione `--oneline` vengono mostrate solo le prime 7 cifre, ma l'identificativo completo è lungo 40 caratteri.

Questo hash viene generato automaticamente da Git in base al contenuto del progetto e garantisce che ogni versione sia univoca, rendendo impossibile modificare retroattivamente un commit senza generarne uno nuovo. Nell'output compare anche il riferimento `HEAD` che rappresenta un puntatore al commit attualmente selezionato, ovvero alla versione del progetto su cui si sta lavorando; nel caso mostrato, `HEAD` coincide con il branch `master` e punta all'ultimo commit. Utilizzando il comando `git checkout <hash commit>` è possibile spostare `HEAD` su un commit specifico rendendo quella versione la base di lavoro corrente e, di conseguenza, i file presenti nella `working directory`, corrisponderanno allo stato salvato in quel commit. Una nota importante è che in genere i file *modified* vengono trasportati da una versione all'altra e questo è un problema perché si rischia di mischiare file appartenenti a versioni diverse, per evitare questa situazione si può usare il comando `git stash -u` che praticamente crea una copia di tutti i file nella `working directory` e nella `staging area` e li mette in uno stack tipo LIFO (Last In, First Out), questo permette di cambiare versione avendo le zone di lavoro pulite. Per ripristinare l'ultimo stato memorizzato nello stack si usa `git stash pop`, anche qui è importante precisare che il ripristino sarebbe meglio effettuarlo avendo `working directory` e `staging area` vuote e anche in corrispondenza dello stesso commit in cui era stato eseguito il salvataggio dello stato. Oltre a questo utilizzo, `git stash` è usato anche per altre operazioni come ad esempio trasportare file modificati da un commit ad un altro (per fare confronti o qualsiasi altra operazione), a tal fine tornano utili i seguenti comandi: `git stash list`, che mostra tutti i salvataggi presenti nello stack (ogni salvataggio è indicizzato con un indice assegnato in modo progressivo), `git stash apply <indice>`, che ripristina uno specifico salvataggio nella posizione corrente e senza rimuoverlo dallo stack, e `git stash drop <indice>`, che elimina un elemento dallo stack.

A questo punto, essendo possibile spostarsi tra diverse versioni, si può osservare un comportamento fondamentale di Git. Se si seleziona un commit precedente, si modificano i file e si crea un nuovo commit, la cronologia non prosegue semplicemente in avanti, ma si genera una nuova linea di sviluppo parallela. Si consideri il seguente output, ottenuto dopo aver effettuato un commit a partire dal `529e5f6`:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti ((5a0319a...))
$ git log --all --oneline --graph
* 5a0319a (HEAD) aggiunto aroldi a storia
| * 8f32569 (master) modificato il file giolitti
| * 2877e77 modificato il file giolitti
| * 987e4ba aggiornato file fibonacci
|/
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

In questo caso si osserva chiaramente la presenza di due rami: uno principale, identificato dal puntatore `master`, e uno secondario, su cui si trova attualmente `HEAD`. Il nuovo commit non è stato aggiunto in coda alla linea principale, ma ha dato origine a un nuovo percorso nella cronologia, questo accade perché ogni commit contiene un riferimento al commit precedente e creando un commit a partire da una versione passata si costruisce una nuova sequenza indipendente che si sviluppa parallelamente a quella già esistente. Si nota inoltre che `HEAD` non punta più a `master`, ma direttamente a un commit, questa situazione è nota come *detached HEAD* e indica che non si sta lavorando su un branch, ma su un commit specifico. Per rendere questa nuova linea di sviluppo persistente e facilmente accessibile (altrimenti sarebbe necessario usare ogni volta hash dell'ultimo commit) è possibile creare un branch usando il comando `git branch <nome del branch>`, una volta creato è possibile spostarsi su di esso e continuare lo sviluppo in modo strutturato, mantenendo separate le diverse linee di evoluzione del progetto. A seguire output di `git log --all --oneline --graph` dopo aver "creato" il nuovo branch e aver spostato `HEAD` su esso.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ git log --all --oneline --graph
* 5a0319a (HEAD -> mattia) aggiunto aroldi a storia
| * 8f32569 (master) modificato il file giolitti
| * 2877e77 modificato il file giolitti
| * 987e4ba aggiornato file fibonacci
|/
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
```

```
* dfcd8f9 initial commit
```

Lo sviluppo tramite branch rappresenta una delle caratteristiche fondamentali di Git e può essere sfruttato in diversi contesti. Uno dei principali è la collaborazione tra più persone sullo stesso archivio: in questo caso, una prassi comune consiste nell'utilizzare un branch dedicato per ciascun collaboratore, permettendo a ognuno di sviluppare in modo indipendente senza interferire direttamente con il lavoro altrui. Le modifiche introdotte nei vari branch vengono poi progressivamente integrate tra loro attraverso operazioni di unione (*merge*), fino a confluire nel ramo principale, generalmente identificato con *master* (o *main* nelle configurazioni più recenti), che rappresenta la versione di riferimento del progetto.

### 1.3. Merge dei branch

Dopo aver introdotto il concetto di branch, è possibile analizzare come e perché questi possano essere uniti, tale operazione prende il nome di *merge* tra branch. Nella maggior parte dei casi, il merge viene utilizzato per integrare un branch secondario (ad esempio sviluppato per una nuova funzionalità) all'interno del branch principale.

Il comando di base è `git merge <branch>`: questa operazione integra i contenuti del branch specificato all'interno del branch corrente (ricordando che il nome di un branch rappresenta un puntatore all'ultimo commit di una determinata linea di sviluppo). L'integrazione può avvenire in due modi:

- **Diretta:** quando le modifiche riguardano parti diverse dei file, il merge viene eseguito automaticamente senza richiedere intervento.
- **Conflittuale:** quando le modifiche interessano le stesse parti di uno o più file, è necessario un intervento manuale.

Il caso diretto è immediato, quello conflittuale richiede qualche attenzione in più, infatti, quando due branch modificano lo stesso punto di un file (ad esempio la stessa riga), Git non è in grado di stabilire quale versione mantenere e richiede quindi una risoluzione manuale. In questi casi Git evidenzia il conflitto direttamente all'interno dei file usando marcatori speciali che delimitano le diverse versioni in conflitto, l'utente deve modificare il file scegliendo cosa mantenere (una delle due versioni oppure una combinazione di entrambe), rimuovere i marcatori e salvare il risultato finale. Corretti i file, è necessario aggiungere questi alla staging area ed effettuare un commit che rappresenta il completamento del merge; i file non coinvolti nei conflitti vengono invece integrati automaticamente.

Vediamo ora un esempio che mostra entrambe le situazioni. L'albero di partenza è il seguente:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (giorgia)
$ git log --all --oneline --graph
* fffb618 (HEAD -> giorgia) aggiunta cartella musica e Verdi
| * 33f6bce (master) modificato il file euclide
| * 8f32569 modificato il file giolitti
| * 2877e77 modificato il file giolitti
| * 987e4ba aggiornato file fibonacci
|/
| * 1ab128a (mattia) modificato file euclide
| * 5a0319a aggiunto aroldi a storia
|/
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

Sono presenti tre branch: uno principale, *master*, e due secondari, *mattia* e *giorgia*. Il contenuto del file `euclide.txt` nei vari branch è il seguente:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema
Atene.
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ git checkout master
```

```
Switched to branch 'master'
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema
Platone
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ git checkout giorgia
Switched to branch 'giorgia'
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (giorgia)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema
```

Nel branch giorgia è inoltre presente una cartella aggiuntiva musica contenente Verdi.txt. Si tenta ora un merge tra master e mattia, è naturale aspettarsi un conflitto sul file euclide.txt in quanto modificato nello stesso punto:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git merge mattia
Auto-merging matematica/euclide.txt
CONFLICT (content): Merge conflict in matematica/euclide.txt
Automatic merge failed; fix conflicts and then commit the result.

snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master|MERGING)
```

durante questa fase, il repository si trova in uno stato intermedio indicato come (master|MERGING), che segnala la presenza di un merge non ancora completato, il file in conflitto appare nel seguente modo:

```
<<<<<<< HEAD
Platone
=====
Atene.
>>>>>>> mattia
```

i marcatori indicano:

- HEAD: versione del branch corrente
- mattia: versione del branch da integrare

Si può quindi risolvere il conflitto mantenendo entrambe le versioni, modificando il file e rimuovendo i marcatori:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master|MERGING)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema.
Platone.
Atene.
```

si eseguono `git add *` e `git commit`, completando il merge. Il risultato è un nuovo commit che unisce le due linee di sviluppo. Successivamente, si può effettuare il merge del branch giorgia in master; in questo caso non si verificano conflitti, poiché le modifiche riguardano file diversi o non sovrapposti:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git merge giorgia
Merge made by the 'ort' strategy.
musica/Verdi.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 musica/Verdi.txt
```

in questo caso Git esegue automaticamente l'integrazione e crea direttamente un commit di merge (eventualmente aprendo l'editor per il messaggio). Al termine delle operazioni, il grafo dei commit mostra chiaramente l'unione delle diverse linee di sviluppo:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git log --all --oneline --graph
*   30d99d9 (HEAD -> master) merge master-giorgia
|\
| * ffb618 (giorgia) aggiuta cartella musica e Verdi
* |   3183e7f fix merge master-mattia
|\ \
| * | 1ab128a (mattia) modificato file euclide
| * | 5a0319a aggiunto aroldi a storia
| | /
* | 33f6bce modificato il file euclide
* | 8f32569 modificato il file giolitti
* | 2877e77 modificato il file giolitti
* | 987e4ba aggiornato file fibonacci
| /
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

L'unico vero branch rimasto è il `master`, gli altri due continuano ad esistere, infatti eseguendo il comando `git branch --list` compaiono nella lista ma concettualmente non ha molto senso tenerli (diciamo che molto dipende dai casi di utilizzo), per eliminarli si usa il comando `git branch -d <nome del branch>`.

# Fonti

- [1] Website Name <OR> I. Surname, I. Surname e I. Surname. *Title of the Website*. 2000. URL: <https://example.com> (visitato il 24/12/2020).